

Report
Of
PacketRadar
Live Network Traffic Monitoring and PCAP Analysis Dashboard

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.
In partial fulfilment of the requirement of degree of
B.Tech CSE



By
Roll Number of Student: CS-23411426
Name of Student: YATIN PARASHAR
Program: B.Tech CSE | Semester: 7th
Batch: 2023 – 2027

2026-27

CONTENTS

Chapter No.	Chapter Name	Page No.
	Cover Page	----
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Internship Completion Certificate	iii
4.	Project Description 4.1 Introduction 4.2 Organization Profile 4.3 Problem Statement 4.4 Project Objectives 4.5 Scope of the Project 4.6 Technologies and Tools Used 4.7 System Architecture 4.8 Methodology 4.9 Expected Outcomes 4.10 Certificates of Completion / Communication Proof	1
5.	Bibliography / References	

CANDIDATE'S DECLARATION

I, Yatin Parashar, Roll Number CS-23411426, do hereby declare that the internship report titled “Packet Radar” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. All the references have been duly quoted and I have not taken as such any material from any other source. I affirm that this report is my own work and has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

Signature: _____

Student Name: Yatin Parashar

Roll No.: CS-23411426

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work. I express my gratitude to my parents for their blessings.

I would also like to thank the faculty of the School of Computer Science and Engineering, IILM University, Greater Noida, for their continuous academic support.

I express my heartfelt gratitude to my parents and family for their constant encouragement and blessings throughout this journey.

Signature: _____

Student Name: Yatin Parashar

Roll No.: CS-23411426

INTERNSHIP COMPLETION CERTIFICATE



4. PROJECT DESCRIPTION

4.1 Introduction

PacketRadar is a full-stack network traffic monitoring and analysis application developed as part of the B.Tech CSE internship/project work. The system combines a real-time packet capture dashboard with an offline PCAP (packet capture file) analysis module, allowing a user to either observe live network activity as it happens or upload a previously recorded capture file for detailed inspection. The application is built with a Python (FastAPI) backend that performs packet capture and analysis, and a React (TypeScript) frontend that visualizes the resulting data through charts, tables, and live-updating panels. The motivation behind the project is to provide a single, unified tool that brings together packet-level visibility, traffic statistics, host discovery, and basic threat detection, which are otherwise available only as separate command-line utilities such as Wireshark, tcpdump, or standalone intrusion-detection scripts. PacketRadar packages these capabilities into a browser-based dashboard that is easier to use for monitoring a local network or reviewing a capture file.

4.2 Organization Profile

Defence Research and Development Organisation (DRDO) is India's premier organization responsible for research and development in defence technologies. The Systems Analysis Group (SAG), a specialized laboratory of DRDO located in Delhi, focuses on research and development in areas such as cryptography, information security, cyber security, and secure communication systems. SAG works on developing advanced technologies to strengthen the security of critical defence information and communication networks. The organization also provides opportunities for students and researchers to gain practical exposure to defence-oriented research and technological development.

4.3 Problem Statement

Monitoring network traffic and identifying suspicious activity typically requires switching between multiple specialized tools: a packet sniffer for live capture, a separate analyzer for reading capture files, and manual inspection or scripting to spot anomalies such as port scans, spoofing attempts, or unusual data transfers. This separation makes it difficult for a student or a small team to get a consolidated, real-time view of what is happening on a network, and it leaves the interpretation of raw packet data almost entirely to the user. There is a need for a lightweight application

that captures packets, organizes them into meaningful statistics, applies basic threat-detection rules, and presents the results through a readable dashboard, without requiring the user to write custom scripts for every task.

4.4 Project Objectives

The main objectives of the project are as follows:

- To capture live network packets on the host machine and stream summarized statistics to a web dashboard in real time.
- To allow a user to upload an existing PCAP file and analyze it offline without needing a live capture.
- To classify captured packets by protocol, source/destination hosts, and application-level ports, and present this information as summary statistics and charts.
- To detect common suspicious traffic patterns — port scanning, ARP spoofing, DNS tunneling, SYN flooding, beaconing, and data exfiltration — using simple, rule-based heuristics.
- To let the user start and stop packet recording and export the recorded traffic as a PCAP file.
- To generate a downloadable PDF report summarizing the findings of an analyzed capture, including charts, tables, and recommendations.

4.5 Scope of the Project

The scope of Packet Radar covers packet capture and analysis at the network and transport layers (Ethernet, ARP, IP, IPv6, TCP, UDP, ICMP, and DNS). The application classifies traffic by protocol and port, tracks top talkers and host-to-host conversations, and applies six independent behavioral detection rules to flag potentially malicious activity. Both a live-monitoring mode and an offline PCAP-analysis mode are supported, and analyzed captures can be exported as a formatted PDF report. The project has some explicitly documented limitations that fall outside its current scope. HTTP and HTTPS traffic is classified only by transport-layer port number and is not decoded at the application layer, so HTTP methods, status codes, and TLS certificate/SNI details are not extracted. ICMP packets are counted but their type and code are not classified. There is no MAC-address vendor (OUI) lookup or reverse-DNS hostname resolution wired into host discovery, so unresolved hosts are shown as unknown. TCP retransmissions and failed connections are identified using simple heuristics (such as repeated bare SYN packets) rather than full TCP stream reassembly. These limitations are surfaced directly in the application's own output rather than hidden, and are listed here as boundaries of the current implementation.

4.6 Technologies and Tools Used

- Frontend: React 18, TypeScript, Vite, Tailwind CSS, React Router, Chart.js (via react-chartjs-2), and the lucide-react icon library.
- Backend: Python, FastAPI, Uvicorn (ASGI server), Pydantic (data validation and schemas), and native WebSockets for live data streaming.
- Packet Capture and Analysis: Scapy, used for sniffing live network interfaces and for parsing PCAP files into structured packet records.
- Report Generation: ReportLab, used to generate the PDF analysis report, and Matplotlib, used to render the charts embedded in that report.

- Supporting Libraries: psutil (system/process information) and python-multipart (handling file uploads such as PCAP files).
- Development Tools: Node.js/npm for the frontend build pipeline, PostCSS and Autoprefixer, and TypeScript's compiler for type checking.

4.7 System Architecture

PacketRadar follows a client-server architecture with a clear separation between the packet-processing backend and the visualization frontend. The backend is organized into distinct layers: a capture layer, a parsing layer, an engines layer, a caching/storage layer, an API layer, and a reporting layer.

The capture layer (`app/capture`) is responsible for sniffing packets from a live network interface using Scapy, resolving hostnames and process information where possible, and running a background watchdog task that restarts the capture thread if it stalls or dies. The parsing layer (`app/parser`) converts raw Scapy packets, whether from a live interface or an uploaded PCAP file, into a common structured representation called a `PacketModel`, covering Ethernet, ARP, IP/IPv6, TCP, UDP, ICMP, and DNS fields.

The engines layer (`app/engines`) consumes these `PacketModel` records and computes the actual insights shown in the dashboard: protocol distribution, host discovery, top talkers, host-to-host conversations, protocol activity over time, DNS analysis, and threat detection. The threat detection engine independently evaluates six behavioral rules — port scan, ARP spoofing, DNS tunneling, SYN flood, beaconing, and data exfiltration — and raises alerts that are also streamed live to the frontend. The caching/storage layer (`app/cache`) holds recent packets, resolved hostnames, and uploaded PCAP analysis results in memory so that API and WebSocket requests can be served quickly without reprocessing the same data repeatedly.

The API layer (`app/api`) exposes REST endpoints for capture control, packet retrieval, PCAP upload/analysis, and PDF report generation, while the WebSocket layer (`app/ws`) pushes live packet and statistics updates to connected clients without requiring the frontend to poll.

The reporting layer (`app/report`) takes the already-computed analysis of a PCAP capture and builds a structured `Report` object, which is then rendered into a multi-section PDF using ReportLab, with Matplotlib-generated charts embedded as images. This layer only reads the already-computed `Report` object and never touches raw packets directly, keeping report generation independent of the analysis engines that

feed it. On the client side, the React frontend communicates with the backend over REST for on-demand data (such as PCAP upload and analysis results) and over WebSockets for continuously updating live statistics and packet streams. The frontend is organized into pages for Live Monitor, PCAP Analyzer, and an About/tech-stack page, each composed of reusable components such as stat cards, tables, charts, and filter bars.

4.8 Methodology

The project was developed and can be described through the following methodology:

- **Packet capture:** Scapy sniffs packets from a chosen network interface in a background thread when running in live mode, or reads an uploaded .pcap/.pcapng file when running in offline mode.
- **Parsing:** every captured packet is passed through the PacketParser, which extracts protocol-specific fields (MAC/IP addresses, ports, DNS query/response data, TCP flags, and so on) into a common PacketModel.
- **Analysis:** the parsed packets are fed into the various engines, each responsible for one kind of derived statistic — protocol distribution, top talkers, host discovery, conversations, protocol timeline, DNS analysis, and threat detection.
- **Serving results:** the API layer exposes the computed results as REST endpoints (for PCAP analysis and on-demand queries) and the WebSocket layer streams incremental updates for live monitoring, using sequence numbers so a newly connected client can also receive recent backlog data.
- **Visualization:** the React frontend consumes these REST and WebSocket endpoints and renders them as KPI cards, tables (hosts, talkers, threats, packets), and Chart.js visualizations, updating live views as new WebSocket messages arrive.
- **Reporting:** for an analyzed PCAP capture, the report_builder module reuses the outputs of the existing engines to assemble a structured Report object, which pdf_generator then renders into a multi-section PDF (cover page, summary, charts, tables, and recommendations) that can be downloaded through the API. **Deployment:** Deployed the backend to Railway and verified the complete flow in a production-like environment.

4.9 Expected Outcomes

Based on the implemented functionality, the expected outcomes of the project are:

- A working web dashboard that displays live network traffic statistics, top talkers, and host discovery information in real time.
- The ability to upload a PCAP file and receive a full breakdown of its traffic, including protocol distribution, DNS activity, host conversations, and a protocol timeline.
- Automatic flagging of six categories of suspicious traffic behavior — port scans, ARP spoofing, DNS tunneling, SYN floods, beaconing, and data exfiltration — with supporting evidence for each alert.
- The ability to start and stop packet recording during a live session and export the recorded traffic as a PCAP file for later analysis.
- A downloadable, formatted PDF report for any analyzed PCAP capture, containing an executive summary, charts, tables, and data-driven recommendations.

4.10 Certificates of Completion and Communication Proof

The Certificate of Internship and Certificate of Training issued by Defence Research and development organization, covering the period 15 June 2026 to 31 July 2026, are attached in Chapter 3 of this report as proof of successful completion of the internship program.

5. BIBLIOGRAPHY / REFERENCES

1. FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/>
2. Scapy Documentation — Packet Manipulation Library for Python. Available at: <https://scapy.readthedocs.io/>
3. React Documentation. Available at: <https://react.dev/>
4. Vite Documentation. Available at: <https://vitejs.dev/>
5. Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>
6. Chart.js Documentation. Available at: <https://www.chartjs.org/docs/latest/>
7. Pydantic Documentation. Available at: <https://docs.pydantic.dev/>
8. ReportLab User Guide — PDF Generation Toolkit for Python. Available at: <https://www.reportlab.com/docs/reportlab-userguide.pdf>
9. Matplotlib Documentation. Available at: <https://matplotlib.org/stable/index.html>